

Plano de Evolução para E-commerce — Vitrine Largo da Ordem

Plano de Evolução para E-commerce — Vitrine Largo da Ordem

Documento de planejamento estratégico para evoluir a vitrine SwiftUI atual (catálogo estático de 12 obras dos artesãos do Largo da Ordem, Curitiba) em um marketplace iOS completo, sustentável e entregável por uma dupla acadêmica da PUCPR.

1. Visão

Onde estamos hoje

A `VitrineLargoOrdem` é um app SwiftUI single-target com:

- `VitrineView` + `DetalhesProdutoView` navegando via `NavigationStack`.
- `VitrineViewModel` com 12 produtos mockados em memória, filtro por busca textual e favoritos voláteis.
- `ProdutoArtesanal` (struct) carregando `isFavorito`, `imagemNome` e formatação de preço no próprio model.
- Zero persistência, zero rede, zero autenticação, zero testes.

É um excelente **showroom acadêmico**, mas estruturalmente está a uma decisão arquitetural de virar um e-commerce real.

Onde queremos chegar

Um marketplace iOS publicável na App Store, focado em artesanato curitibano, com:

- Catálogo dinâmico mantido pelos próprios artesãos.
- Carrinho e checkout funcionais com pagamento real (cartão + Apple Pay + Pix).
- Pós-venda completo (status do pedido, rastreio, avaliações).
- Identidade do usuário com Sign in with Apple, perfil e LGPD compliance.
- Distribuição contínua via TestFlight + App Store, com observabilidade básica.

O caminho é **incremental e demonstrável a cada fase** — cada release tem valor isolado e cabe na capacidade de 2 desenvolvedores.

2. Princípios Norteadores

#	Princípio	O que significa na prática
1	Offline-first	Catálogo, favoritos e carrinho funcionam sem conexão. Sync é eventual, nunca bloqueante.
2	Acessibilidade desde o primeiro pixel	Dynamic Type, VoiceOver, contraste AA, <code>accessibilityIdentifier</code> em fluxos críticos. Audit no CI a partir da V2.
3	MVP demonstrável > arquitetura perfeita	Nada de TCA, Coordinators barrocos ou microserviços. <code>@Observable</code> + Repository + Service. Refatorar sob demanda.
4	Separação de mock e produção via Repository	Toda fonte de dados é protocolada. Demos acadêmicas rodam 100% local; produção troca a implementação.
5	Identidade visual coerente	Design tokens (cores, tipografia, espaçamento) centralizados desde o MVP. Sem isso, 70+ features viram colcha de retalhos.
6	Privacidade e LGPD como requisito, não feature	Consentimento explícito, exclusão de conta, ATT antes de qualquer SDK de tracking.

3. Decisões Técnicas Estratégicas

Estas decisões precedem qualquer linha de código nova. Mudá-las depois custa caro.

Decisão	Escolha	Justificativa
Backend	Supabase (Postgres + Auth + Storage + Edge Functions)	SQL nativo, RLS granular, open-source, sem lock-in. Studio resolve admin sem precisar de portal web próprio na V1. Firebase ganharia em push, mas perderia em modelagem relacional (crítico para pedidos/itens).
Persistência local	SwiftData (<code>@Model</code>) com versionamento explícito	Alinhado com iOS 17+ do projeto, sintaxe enxuta. Encapsular tudo atrás de <code>Repository</code> para mitigar bugs de migration trocando por Core Data se necessário.
Arquitetura de estado	<code>@Observable</code> (iOS 17) + Repository + Service, MVVM leve	TCA é overkill para 2 devs. Coordinators só na V2 se navegação ficar profunda.
Cache de imagens	Nuke (não Kingfisher)	Melhor integração SwiftUI (<code>LazyImage</code>), API moderna async/await, footprint menor. Kingfisher ainda carrega legado UIKit.
PSP (gateway)	Mercado Pago Mobile SDK	Pix nativo BR, melhor onboarding para pequenos comerciantes/artesãos, suporte a Apple Pay. Stripe seria escolha se foco fosse internacional.
Auth	Sign in with Apple + Supabase Auth (JWT)	Cumprir Guideline 4.8 da Apple. E-mail/senha como fallback. Google/Facebook só na V3 se houver demanda.
Analytics	TelemetryDeck	Privacy-first, sem PII, sem prompt ATT obrigatório, alinhado a LGPD. Mixpanel/Firebase ficam para depois se precisar de funis complexos.
Crash reporting	Sentry	Open-source friendly, captura erros não-fatais e breadcrumbs. Crashlytics implicaria carregar Firebase sem usar o resto.
DTO ↔ Domain ↔ SwiftData	Camadas separadas com <code>Mapper</code> explícito	Modelo de API muda, schema de DB muda; isolá-los evita refactor em cascata.
Navegação	<code>NavigationStack</code> tipado com enum <code>Route</code> por feature	Suporta deep linking para push e universal links sem precisar de Coordinator central.
Modularização	Monolito Xcode com pastas por feature; SPM packages só se ultrapassar 30k LOC	2 devs não justificam overhead de modularização agora. Pasta-por-feature já antecipa split futuro.
Testes	Swift Testing (<code>@Test</code>) para unit, XCUITest para smoke	Swift Testing é a aposta da Apple e tem melhor ergonomia que XCTest.
CI/CD	GitHub Actions + Fastlane (match, gym, pilot)	Padrão de mercado, runners macOS disponíveis no plano grátis com limites aceitáveis para 2 devs.
Modelo de negócio	Marketplace sem split payment no V1; comissão calculada off-app	Stripe Connect / split de PSP exigem CNPJ, contrato e compliance fiscal — fora do escopo acadêmico. Conta única recebe e repassa manualmente até V3.

4. Roadmap por Fase

MVP — Vitrine navegável e carrinho local (4–6 semanas)

Objetivo: rodar 100% offline, demonstrável em sala, sem backend. Entregar a **sensação de e-commerce** sem custo de infra.

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
0	Design tokens + estrutura de pastas por feature	must	baixo	Criar <code>DesignSystem/</code> (Color, Typography, Spacing); reorganizar em <code>Features/Vitrine</code> , <code>Features/Carrinho</code> , <code>Core/</code>	SwiftUI, AssetCatalog, ShapeStyle
1	Modelagem SwiftData + seed das 12 obras	must	médio	<code>ProdutoArtesanal</code> vira <code>@Model</code> ; criar <code>ModelContainer</code> no <code>App</code> ; remover mock estático do ViewModel	SwiftData, JSON seed em bundle
2	Repository pattern + protocolos para mock/SwiftData	must	baixo	<code>ProdutoRepository</code> , <code>FavoritosRepository</code> , <code>CarrinhoRepository</code> protocolados	Swift, DI manual
3	Categorias com chips horizontais + ordenação	must	baixo	Enum <code>CategoriaArtesanal</code> ; <code>@Published</code> no ViewModel	SwiftUI ScrollView horizontal, SF Symbols
4	Favoritos persistentes (sem login)	must	baixo	<code>FavoritosService</code> consumindo SwiftData; remover <code>isFavorito</code> do model de domínio	SwiftData, <code>@Observable</code>
5	Carrinho local (add/remove/quantidade) + persistência	must	médio	<code>CarrinhoViewModel</code> , <code>ItemCarrinho</code> ; TabView raiz substituindo UINavigationController único	SwiftUI TabView, SwiftData, haptics
6	Resumo financeiro (subtotal/frete fixo/total)	must	baixo	<code>PrecoFormatter</code> centralizado em <code>Core/Formatters</code> ; <code>ResumoPedidoView</code> reusável	NumberFormatter, Decimal
7	Galeria múltiplas imagens com zoom	should	médio	<code>imagemNome</code> vira <code>[URL]</code> ; <code>GaleriaProdutoView</code> paginada	TabView .page, MagnificationGesture
8	Skeleton loading + empty states	should	baixo	<code>SkeletonView</code> + <code>ContentUnavailableView</code> padronizados	LinearGradient animado, SwiftUI nativo
9	Indicador de estoque / esgotado	must	baixo	Adicionar <code>estoque: Int</code> ao model; badge no card	SwiftUI Badge + overlay
10	Testes unitários de ViewModels	must	baixo	Target <code>VitrineLargoOrdemTests</code>	Swift Testing, mocks manuais

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
				com Swift Testing; mocks via protocolo	

Critérios de pronto (MVP):

- App roda do cold start ao checkout local sem rede.
- Reabrir o app preserva favoritos, carrinho e histórico.
- Cobertura $\geq 70\%$ nos ViewModels do carrinho e da vitrine.
- Sem warnings no Xcode, sem força-unwrap em código de produção.
- Demonstração de fluxo completo em sala em < 3 minutos.

V1 — Backend, identidade e checkout simulado (6–8 semanas)

Objetivo: deixar o app pronto para TestFlight fechado com artesãos parceiros. Pagamento ainda **mockado**; o resto é real.

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
1	Setup Supabase (Postgres + Auth + Storage)	must	alto	Schema relacional Produto/Artesao/Pedido/User; RLS; bucket de imagens com signed URLs	Supabase, supabase-swift, PostgREST
2	Camada Networking + DTO/Mapper	must	médio	<code>APIClient</code> com async/await + interceptor de token; Codable DTOs separados dos models	URLSession, Codable
3	Sign in with Apple consolidado (UI + JWT)	must	médio	<code>AuthService</code> Observable; <code>Keychain</code> para refresh token; AuthGate no App	AuthenticationServices, KeychainAccess
4	Cadastro/login e-mail + recuperação de senha	must	médio	<code>EmailAuthService</code> ; Universal Links para reset	Supabase Auth, Associated Domains
5	Perfil + endereços (CRUD + ViaCEP)	must	médio	TabView ganha aba Perfil; <code>EnderecoRepository</code>	ViaCEP, PhotosPicker, Storage
6	Exclusão de conta + portabilidade (LGPD)	must	médio	Edge Function <code>deleteAccount</code> com soft delete; export JSON	Supabase Edge Functions, pg_cron
7	Checkout multi-step com pagamento mock	must	alto	<code>CheckoutViewModel</code> , <code>NavigationStack(path:)</code> tipado; etapas Endereço/Pagamento/Revisão	SwiftUI NavigationStack, SwiftData rascunho
8	Tela de confirmação de pedido	must	baixo	<code>PedidoConfirmadoView</code> , limpar carrinho, gravar em histórico	SwiftUI, ShareLink
9	Meus Pedidos + Status timeline (mock local)	must	médio	<code>PedidosViewModel</code> , <code>TimelineStatusView</code> reusável	SwiftUI Canvas/Shape, SwiftData
10	Cache de imagens com Nuke + CDN Supabase Transformations	must	baixo	Substituir <code>AsyncImage</code> por <code>LazyImage</code> ; URLs com variantes	Nuke, Supabase Storage
11	Crashlytics/Sentry + logs estruturados (OSLog)	must	baixo	<code>Logger</code> por subsystem; SDK no <code>App.init</code>	Sentry, OSLog
12	TelemetryDeck + consent gate + ATT	must	baixo	<code>AnalyticsService</code> protocolado; eventos enum-typed; prompt ATT contextual	TelemetryDeck, AppTrackingTransparency

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
13	Compliance LGPD + política de privacidade no app	must	médio	<code>ConsentManager</code> ; webview da política; <code>PrivacyInfo.xcprivacy</code>	SwiftUI, Info.plist
14	CI/CD GitHub Actions + Fastlane + TestFlight	must	médio	Workflows ci.yml e release.yml; secrets do App Store Connect API	GitHub Actions, Fastlane (match/gym/pilot)
15	App Store listing + ASO inicial	must	médio	Screenshots localizados via fastlane snapshot; nutrition label	Fastlane deliver, ASO tools
16	Onboarding inicial (3 telas)	should	baixo	Apresenta proposta do Largo da Ordem; mostrado uma vez	SwiftUI TabView .page, AppStorage

Critérios de pronto (V1):

- TestFlight aberto para grupo externo (artesãos + early adopters).
- Auth com Apple + e-mail funcionando em produção.
- Checkout finaliza pedido com pagamento mockado, pedido aparece em "Meus Pedidos".
- Pipeline CI verde em todos os PRs; release automatizado para TestFlight em merge para `main`.
- Crash-free rate > 99% após 1 semana com 20 usuários.

V2 — Monetização e relacionamento (6–8 semanas)

Objetivo: transformar app em e-commerce com receita real e engajamento contínuo.

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
1	Integração Mercado Pago + cartão + Pix	must	alto	<code>PagamentoService</code> real; webhooks via Edge Function atualizam pedido	Mercado Pago iOS SDK
2	Apple Pay (PassKit)	must	médio	<code>PKPaymentRequest</code> ; merchant ID; token enviado ao gateway	PassKit, MP SDK
3	Gestão de cartões salvos (tokenizados)	should	médio	<code>PaymentMethodsView</code> ; nunca persistir PAN	MP tokens, Keychain
4	Push notifications de status do pedido	must	alto	Capability Push; APNs via FCM; categories com actions; deep link	UserNotifications, FCM
5	E-mail transacional (confirmação + reenvio)	should	médio	Webhook de pedido dispara template; endpoint reenviar	Resend ou SendGrid, MJML
6	Avaliação pós-compra (estrelas + foto + comentário)	should	médio	Acionada por push 48h após entrega; upload de fotos comprimidas	PhotosUI, Supabase Storage, AVFoundation
7	Exibição de reviews na PDP	should	médio	Seção em <code>DetalhesProdutoView</code> com média + lista paginada	SwiftUI, <code>@Query</code> SwiftData
8	Painel admin via Supabase Studio customizado	should	médio	Views/políticas para artesãos editarem só seus produtos	Supabase Studio, RLS
9	Recuperação de carrinho abandonado	should	médio	Job no backend agenda push + e-mail; deep link para checkout	Cloud Tasks, APNs
10	Filtros avançados (preço, categoria, artesão)	should	médio	<code>FiltrosSheetView</code> ; <code>FiltrosVitrine</code> codável; persistência sessão	SwiftUI Sheet, Slider
11	Busca com sugestões e histórico	should	baixo	<code>.searchSuggestions</code> agrupado; NSAttributedString highlight	SwiftUI nativo, AppStorage
12	Snapshot tests + i18n PT/EN	should	médio	swift-snapshot-testing; migração de strings para <code>.xcstrings</code>	Point-Free snapshot, String Catalogs

#	Feature	Prioridade	Esforço	Mudanças principais	Stack
13	Autenticação biométrica + preferências de notif	should	baixo	<code>BiometricAuthService</code> ; tela de preferências granular	LocalAuthentication, Keychain
14	Auditoria de a11y no CI	should	médio	<code>performAccessibilityAudit</code> no XCUITest; falha PR se quebrar contraste	XCUITest iOS 17

Critérios de pronto (V2):

- App processa pagamentos reais com aprovação e webhook atualizando pedido.
- Push notification chega em < 30s após mudança de status.
- $\geq 30\%$ dos pedidos entregues recebem avaliação.
- A11y audit verde no CI; suporte completo a Dynamic Type e Dark Mode.
- App Store listing publicado, disponível para download geral.

V3 — Diferenciação e escala (backlog futuro)

Objetivo: features que diferenciam o produto, mas dependem de tração e dados reais para fazerem sentido.

Feature	Por que adiar	Pré-requisito
Chat 1:1 com artesão	Caro de operar, sem demanda validada	Base de usuários > 500
Devolução/troca (RMA)	Logística reversa complexa, pouco volume no início	Pedidos > 100/mês
Recomendações personalizadas	Precisa de histórico denso de eventos	Catálogo > 100 itens, 1k+ sessões
Programa de fidelidade + referral	Só faz sentido com recompra	Taxa de recompra mensurável
A/B testing + feature flags	Tráfego mínimo para significância estatística	Usuários ativos > 1k/semana
Coleções curadas + artesão em destaque	Requer time editorial	Catálogo > 50 artesãos
Rastreo Correios integrado	API instável; usuário pode copiar código	Volume de tickets de suporte sobre rastreo
Login social Google/Facebook	Sign in with Apple + e-mail cobrem 95%	Pesquisa de fricção de login
Split payment (Stripe Connect)	Implica CNPJ, contratos, compliance fiscal	Modelo de negócio formalizado

5. Mudanças de Arquitetura no Código Atual

Estrutura de pastas evoluída

```
VitrineLargoOrdem/
├── App/
│   ├── VitrineLargoOrdemApp.swift
│   ├── AppDependencies.swift           // composição raiz, injeção
│   └── AuthGate.swift                 // V1+: roteamento auth
├── DesignSystem/
│   ├── Colors.swift
│   ├── Typography.swift
│   ├── Spacing.swift
│   └── Components/                   // ChipView, EstrelasView, SkeletonView
├── Core/
│   ├── Networking/                  // V1+: APIClient, Interceptors
│   ├── Persistence/                // ModelContainer, migrations
│   ├── Formatters/                 // PrecoFormatter, DataFormatter
│   ├── Logging/                    // V1+: Logger por subsystem
│   └── Analytics/                   // V1+: protocolo + TelemetryDeck
├── Features/
│   ├── Vitrine/
│   │   ├── Models/                 // ProdutoArtesanal @Model
│   │   ├── Repositories/           // ProdutoRepository (proto + impls)
│   │   ├── ViewModels/             // VitrineViewModel @Observable
│   │   └── Views/                   // VitrineView, DetalhesProdutoView, GaleriaView
│   ├── Favoritos/
│   ├── Carrinho/                   // MVP
│   ├── Checkout/                   // V1
│   ├── Pedidos/                    // V1
│   ├── Identidade/                 // V1: Auth, Perfil, Enderecos, LGPD
│   ├── Pagamento/                 // V2
│   ├── Reviews/                    // V2
│   └── Notificacoes/               // V2
├── Resources/
│   ├── Assets.xcassets
│   ├── Localizable.xcstrings        // V2
│   └── Seed/produtos.json           // MVP: dados iniciais
└── PrivacyInfo.xcprivacy             // V1
```

Refatorações concretas vindas do código atual

Hoje	Depois	Quando
<code>ProdutoArtesanal</code> struct com <code>isFavorito</code> e <code>imagemNome</code>	<code>@Model</code> SwiftData com <code>[URL]</code> para imagens; favorito vira coleção separada	MVP
<code>VitrineViewModel</code> com <code>produtos</code> hardcoded	<code>VitrineViewModel</code> recebe <code>ProdutoRepository</code> por DI	MVP
<code>NavigationStack</code> único em <code>VitrineLargoOrdemApp</code>	<code>TabView</code> raiz (Vitrine, Carrinho, Pedidos, Perfil) com <code>NavigationStack</code> por aba	MVP (Vitrine+Carrinho) → V1 (todas)
Formatação de preço no model	<code>PrecoFormatter</code> em <code>Core/Formatters</code> usando <code>Locale.current</code> e <code>Decimal</code>	MVP
Sem testes	Target <code>VitrineLargoOrdemTests</code> com Swift Testing; mocks de Repository	MVP
Sem design system	<code>DesignSystem/</code> com tokens, componentes base reutilizados em todas as features	MVP

6. Riscos e Mitigações

#	Risco	Probabilidade	Impacto	Mitigação
1	Migração SwiftData quebra dados de TestFlight quando schema mudar (ex: adicionar campo a Pedido)	Alta	Alto	Versionar schema desde o primeiro <code>@Model</code> ; testar migrations em CI; manter <code>Repository</code> para isolar e poder reverter para Core Data + JSON em último caso.
2	Direitos autorais sobre fotos dos produtos — sem contrato com artesãos, app não pode ir ao ar	Alta	Crítico	Bloquear release público até ter termo de cessão de imagem assinado por cada artesão; usar apenas mocks/CC0 em demos acadêmicas.
3	Aprovação App Store negada por falta de exclusão de conta ou login real funcional (Guidelines 5.1.1(v) e 4.8)	Média	Alto	Tratar como feature must da V1; testar submissão de build interno antes de TestFlight externo.
4	Capacidade de 2 alunos vs escopo de marketplace — risco de não entregar V1 antes do prazo acadêmico	Alta	Alto	Roadmap enxuto (cortou 47 features das 71); checkpoints semanais; "definição de pronto" rigorosa; preferir adiar feature a entregar meio-pronta.
5	PSP exige CNPJ, contrato e split payment que aluno não consegue	Alta	Médio	V2 usa conta única (sem split); repasse manual; documentar dívida fiscal. Avaliar Stripe Express ou lugu se modelo de marketplace for formalizado em V3.

7. Próximos Passos Imediatos

1. **Criar ADR-001** registrando as 14 decisões técnicas da seção 3 (Supabase, SwiftData, Mercado Pago, Nuke, `@Observable`, etc.). Sem isso, dupla diverge nas primeiras semanas.
 2. **Modelar schema de dados** (Produto, Artesão, User, Pedido, ItemPedido, Endereço, Favorito, Review) em um diagrama único; validar contra fluxos MVP/V1/V2 antes de criar `@Model` s.
 3. **Reestruturar pastas para layout por feature** e introduzir `DesignSystem/` com cores e tipografia — refactor mecânico de baixo risco que destrava todo o resto.
 4. **Migrar `ProdutoArtesanal` para `@Model` SwiftData** com seed JSON dos 12 produtos atuais; primeiro PR mensurável do MVP.
 5. **Configurar GitHub Actions com lint + build + testes** desde o MVP (mesmo com 1 teste). Adicionar CI/CD tardiamente é doloroso; adicionar cedo é trivial.
-

Conclusão executiva: o caminho do showroom atual a um e-commerce real cabe em 4 fases e ~50 features priorizadas (das 71 originais, ~12 eram redundantes e ~10 estavam em fase errada). O MVP entrega valor sem backend, a V1 destrava monetização futura ao fechar identidade e infraestrutura, a V2 traz receita e engajamento, e a V3 fica como backlog dirigido por dados reais. As 14 decisões técnicas da seção 3 são o gate antes da primeira linha de código nova — fechá-las é o passo zero.